SLOs, Error Budgets, and Burn Rate, Worked Out by Hand

How "be reliable" becomes a *spendable* number —
every minute, every failed request, and every burn rate counted by hand.

What this document does. It takes the one launch gate from Module 4.8 — the production-readiness audit that commits the agent to a $\geq 95\%$ **trajectory success rate** with an *error budget over a rolling 30 days* — and turns each abstract promise into arithmetic. We fix one agent serving 1,000,000 trajectories a month, compute the downtime it is *allowed* at several availability targets, convert the success-rate SLO into a budget of *failed requests*, and then track the **burn rate** day by day until it trips an alert and a deploy freeze. Nothing is hand-waved. Every number below is reproduced by the small Python program in Appendix A, so the figures are exact and self-consistent.

Where this sits. This is **Topic 1 of Module 4.8** — the math under the "SLOs & SLIs" section and the SLO-program rules ("burn $> 50\%$ mid-window $\rightarrow$ freeze risky ramps; budget exhausts $\rightarrow$ all rollouts pause"). Its companions: Module 4.2 (observability — the dashboards that *measure* the SLIs feeding these formulas) and Module 4.5 (versioning & rollout — the "risky ramps" that a burning budget freezes).

Concepts covered, in order: availability SLO $\rightarrow$ allowed downtime $\cdot$ the identity downtime $= (1-A)\cdot\text{period}$ $\cdot$ error budget $= (1-\text{SLO})\cdot\text{requests}$ $\cdot$ burn rate as consumed-budget over elapsed-time $\cdot$ burn > 1 means early exhaustion $\cdot$ multi-window alerting (fast burn vs. slow burn) $\cdot$ tying exhaustion to a SEV-1 freeze decision $\cdot$ the budget as a speedometer.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

1 The setup: one agent, one month, counted by hand

An SLI (Service Level *Indicator*) *measures* something — "what fraction of trajectories succeeded today?" An SLO (Service Level *Objective*) *commits* to a floor on that indicator — "$\geq 95\%$, over a rolling 30 days." The gap between the objective and 100% is not slack to be ashamed of; it is a **budget** you are allowed to spend. This whole document is the arithmetic of that budget: how big it is, how fast you are spending it, and what happens when the meter hits zero.

The quantities. We pin one agent and one month so every sum is followable on paper. The numbers are illustrative but round; the mechanics are identical at any scale.

Quantity	Symbol	Value here
Length of the SLO window (30-day month) $(30 \times 24 \times 60)$	W	43,200 min
Trajectories served in the window	R	1,000,000
Success-rate objective	SLO	99.9%
Availability targets compared	A	99.5/99.9/99.95%
Revenue per minute of full uptime	r	\$200 / min
Elapsed at the check-in	f	10 days $= \frac{1}{3}W$

Notation. A is an availability fraction (uptime). SLO here is a success-rate fraction. The *error budget* E is a count of failed requests. The *elapsed fraction* $f \in (0, 1]$ is how far through the window we are. The *burn rate* β is dimensionless — budget spent per unit of window spent.

Intuition

"Be reliable" is not actionable; you cannot put it on a dashboard or page someone at 3am. An error budget converts it into a *quantity of failures you are permitted to ship* this window — a wallet. Once reliability is money in a wallet, every later decision (freeze the rollout? page the on-call? call it a SEV-1?) becomes the same simple question any spender asks: how much is left, and how fast is it going?

2 Step 1 — an availability SLO becomes allowed downtime

The cleanest budget to see first is *time*. If you promise availability A over a window of length W, then the downtime you are allowed is whatever is left of the window after the promised uptime:

$$D(A) \;=\; (1 - A) \cdot W. \tag{1}$$

That is the entire identity — "the budget is the part you did *not* promise." For a 30-day month, $W = 30 \times 24 \times 60 = 43{,}200$ minutes. Working the three common targets:

Target A	$1 - A$	$D(A) = (1 - A)\,W$	In human terms
99.5%	0.005	216.0 min	$\sim$ 3.6 hours / month
99.9%	0.001	43.2 min	$\sim$ 43 minutes / month
99.95%	0.0005	21.6 min	$\sim$ 22 minutes / month

Sample check, $A = 99.9\%$: $1 - A = 0.001$, so $D = 0.001 \times 43{,}200 = 43.2$ min. ✓ Notice the leverage: moving from 99.5% to 99.95% — adding two "nines" worth of ambition — shrinks the monthly downtime budget *tenfold*, from 216 min to 21.6 min. Each extra nine is an order of magnitude less room to fail.

Mini-example — this operation in isolation

The factor that does all the work is $1 - A$, the *complement*. At $A = 99.9\%$ the complement is 0.001; multiply by the 43,200-minute window and you get 43.2 minutes. There is no other step. If you ever see "three nines = about 43 minutes a month," it is just $0.001 \times 43{,}200$

— and "four nines = about 4.3 minutes" is the same line with one more zero. The whole availability table is one multiplication, repeated.

3 Step 2 — a success-rate SLO becomes an error budget

Agents rarely fail by being *down*; they fail by returning a *wrong* trajectory while perfectly up. So the budget that bites for an agent is counted in *failed requests*, not minutes. Same shape, different unit — spend the complement of the SLO over the request volume:

$$\boxed{E \;=\; (1 - \mathrm{SLO}) \cdot R} \tag{2}$$

For the module's own commitment, an SLO of 99.9% over $R = 1{,}000{,}000$ trajectories a month:

$$E \;=\; (1 - 0.999) \times 1{,}000{,}000 \;=\; 0.001 \times 1{,}000{,}000 \;=\; 1{,}000 \text{ failed trajectories.}$$

So the agent is *permitted* a thousand failures this month — not as a target, but as the line past which it has broken its promise. ✓ That single number, $E = 1000$, is the wallet we spend for the rest of the document.

Intuition

The error budget reframes failures from "shameful, eliminate them all" to "budgeted, spend them wisely." A thousand failures is not a disaster to hide; it is the headroom that lets you ship a risky rollout, run an experiment, or tolerate a flaky upstream — *as long as you do not overspend*. A team with budget left can move fast. A team with budget gone must stop. The number turns reliability from an argument into an account balance.

4 Step 3 — burn rate: the speedometer on the budget

Knowing the budget is 1000 tells you nothing about whether you are in trouble — 1000 failures spread evenly across the month is fine, but 1000 in a day is a fire. You need a *rate*. Burn rate compares the *fraction of budget already spent* against the *fraction of the window already elapsed*:

$$\beta \;=\; \frac{\text{errors so far} \,/\, E}{f} \;=\; \frac{\text{budget consumed}}{\text{time elapsed}}, \tag{3}$$

where f is the elapsed fraction of the window. The reading is immediate: $\beta = 1$ means you are spending exactly on pace to finish the budget precisely as the window closes. $\beta > 1$ means you will run out *early*; $\beta < 1$ means you will coast in with budget to spare.

Walk our agent forward. Ten days into the 30-day month, the elapsed fraction is $f = 10/30 = \frac{1}{3} \approx 0.333$. Suppose the dashboard shows 300 failures so far:

$$\text{consumed} = \frac{300}{1000} = 0.30, \qquad \beta = \frac{0.30}{0.333\ldots} = 0.90.$$

Burn rate $0.90 < 1$: spending 30% of the wallet in the first third of the month is *just under* pace. At this rate the projected month-end total is $300/0.333\ldots = 900$ failures — under the 1000 budget, so we land safely. ✓ Now suppose instead 600 failures by day 10:

$$\text{consumed} = \frac{600}{1000} = 0.60, \qquad \beta = \frac{0.60}{0.333\ldots} = 1.80.$$

Burn rate $1.80 > 1$: at this pace the projected month-end total is $600/0.333\ldots = 1800$ failures — nearly *double* the budget. The budget will be empty around day 17, not day 30. This is the alert. ✓

Day	f	errors	consumed	β	verdict
10	0.333	300	0.30	0.90	under pace — OK
10	0.333	600	0.60	1.80	over pace — **alert**

Mini-example — this operation in isolation

Burn rate is just two fractions divided. Top: how much of the wallet is gone ($\frac{600}{1000} = 0.6$). Bottom: how much of the month is gone ($\frac{10}{30} = 0.333$). Their ratio, $0.6/0.333 = 1.8$, *is* the burn rate. The unit cancels to "budget-spent per window-spent." A clock that reads 1.0 means you and the calendar are tied; 1.8 means the spending is running 1.8× ahead of the clock, and will cross the finish line long before the month does.

5 Step 4 — multi-window alerting: fast burn vs. slow burn

One burn-rate number over one window is too blunt. Measure over the *whole month* and a sudden outage is invisible until it is too late; measure over *one minute* and every blip pages someone. Mature SLO alerting therefore watches **two windows at once**.

Slow burn is the long window (the whole 30 days, as above). A β creeping past 1 here means a chronic, low-grade quality regression — not an emergency, but a budget that will be gone by month's end. It alerts gently: freeze risky ramps, open a ticket.

Fast burn is a short window (say one hour). It catches an acute outage that would devour the whole month's budget in an afternoon. One hour is $f = 1/(30 \times 24) \approx 0.00139$ of the month. Suppose 50 failures land in that single hour:

$$\text{consumed} = \frac{50}{1000} = 0.05, \qquad \beta = \frac{0.05}{0.00139} \approx 36.$$

A burn rate of **36** means you are spending the budget 36× faster than sustainable: at 50 failures/hour the entire 1000-failure wallet is gone in $1000/50 = 20$ hours — under a day. ✓ That is a page-someone-now signal, not a file-a-ticket one.

Watch out

Do not run only one window. A single 30-day burn-rate alarm is *slow to fire and slow to clear*: an outage that burns half the month's budget in an hour barely moves the 30-day average, so the page comes late — and once it does fire, it keeps firing for days after you have fixed the problem, because the spent budget stays spent in the long window. Pair a fast window (fires on acute outages, clears quickly) with a slow window (catches chronic drift). The module's two rules map exactly onto this: *burn* $> 50\%$ *mid-window* is the slow-window trigger; an exhausted budget is the floor both windows defend.

6 Step 5 — from an empty wallet to a freeze decision

The burn rate is a speedometer; the freeze is the brake it triggers. The module states the policy in two lines, and our numbers make each one bite:

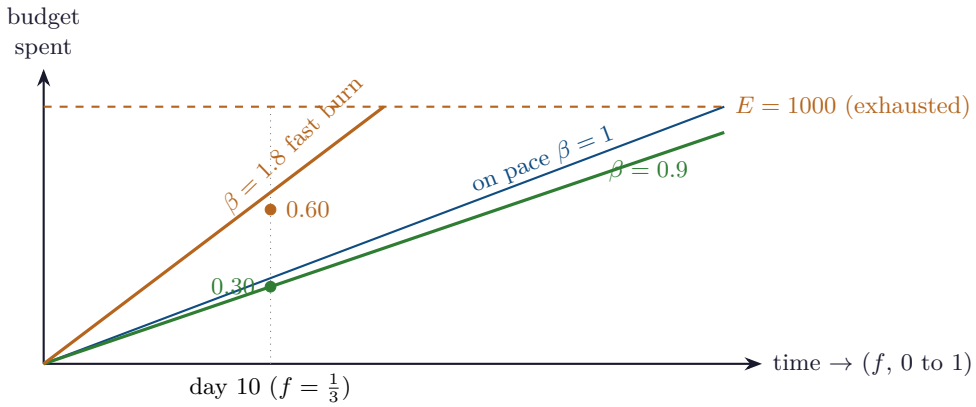
- **Budget burns $> 50\%$ mid-window ⇒ freeze risky ramps.** Our day-10, 600-error case has consumed 60% of the budget at the one-third mark — past the 50% line, well before mid-window. New rollout ramps from Module 4.5 freeze; you stop *adding* risk while the budget is bleeding.
- **Budget exhausts ⇒ all rollouts pause.** When consumed reaches $E = 1000$, the wallet is empty: every ramp halts and SREs plus product agree a recovery plan. There is no discretion left to spend.

This is also where severity attaches. A SEV-1 in the module is "real-world harm, regulatory exposure, or a majority of users affected" — and its cost is naturally read *per minute*. If the agent's failures are an outage burning the time budget, that budget has a price tag: at $r = \$200$ of revenue per minute of uptime, the full month's downtime allowances are worth

Target A	full-budget downtime	revenue at risk $D \cdot r$
99.5%	216.0 min	\$43,200
99.9%	43.2 min	\$8,640
99.95%	21.6 min	\$4,320

Sample check, 99.9%: 43.2 min × \$200/min = \$8,640. ✓ Reading the budget in dollars is what makes the freeze defensible to a SEV-1 war room: "pausing the rollout" stops costing an argument once "not pausing" has a dollar figure per minute attached.

7 The burn, drawn



The diagonal is the sustainable pace, $\beta = 1$: spend the whole budget exactly as the window closes. The green line ($\beta = 0.9$) stays below it and coasts in with room to spare. The orange line ($\beta = 1.8$) is steeper than the diagonal and slams into the dashed ceiling around the window's midpoint — the budget runs out early. Burn rate is simply *the slope of your line divided by the slope of the diagonal.*

8 SLO cheat-sheet

Allowed downtime (time budget)	$D(A) = (1 - A)\,W$
Error budget (failure budget)	$E = (1 - \text{SLO})\,R$
Budget consumed by step k	errors_k / E
Elapsed fraction of window	$f = (\text{time elapsed})/W$
Burn rate	$\beta = \dfrac{\text{errors}/E}{f}$
On pace / safe / alert	$\beta = 1$ / $\beta < 1$ / $\beta > 1$
Projected end-of-window errors	errors/f
Freeze rule (slow window)	consumed $> 50\%$ before mid-window
Pause rule	consumed $\geq E$ (budget exhausted)
Revenue at risk	$D(A) \cdot r$

9 An honest word about these numbers

The constants here are illustrative round numbers, chosen so the sums fall out cleanly by hand: $R = 1{,}000{,}000$ requests, a 99.9% SLO giving a tidy $E = 1000$, a check-in at exactly one-third of the month, and \$200/minute of revenue. Your real values will not be round — your request volume drifts day to day, your window is a *rolling* 30 days (not a fixed calendar month, so f and E slide continuously), and a single trajectory failure is rarely worth a clean per-minute dollar figure. The module itself commits to $\geq 95\%$ success, not 99.9%; we used 99.9% only because it yields a budget of exactly 1000 to count against.

What is *not* illustrative is the structure. The budget is always the complement of the objective times the volume, $D = (1 - A)W$ and $E = (1 - \text{SLO})R$. Burn rate is always consumed-over-elapsed, $\beta = (\text{errors}/E)/f$, and $\beta > 1$ always means early exhaustion. Fast and slow windows always trade off late-firing against noise. Change the constants and the dollar figures move; the shape of the wallet, the speedometer, and the brake does not. That shape is what makes "be reliable" something you can actually operate.

Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the downtime table, the error budget, the day-10 burn rates, the fast-burn figure, and the revenue-at-risk table; change `SLO`, `REQ_PER_MONTH`, or `REV_PER_MIN` to explore your own agent's reliability economics.

```
# slo_budget.py -- reproduces every number in
# "SLOs, Error Budgets, and Burn Rate, by Hand".

# ---------- constants ----------
PERIOD_MIN = 30 * 24 * 60 # minutes in a 30-day month = 43,200
REQ_PER_MONTH = 1_000_000 # trajectories served per month
SLO = 0.999 # success-rate objective (99.9%)
REV_PER_MIN = 200 # $ of revenue per minute of full uptime

# ---------- 1. availability -> allowed downtime ----------
```

```python
def downtime_min(A): # (1 - A) * period
    return (1 - A) * PERIOD_MIN

print("== allowed downtime per 30-day month ==")
for A in (0.995, 0.999, 0.9995):
    print(f" A={A*100:6.2f}% downtime = {downtime_min(A):7.1f} min")

# ---------- 2. error budget ----------
# round-number form: budget = (1 - SLO) * requests
error_budget = round((1 - SLO) * REQ_PER_MONTH)
print(f"== error budget == {error_budget} failed trajectories / month")

# ---------- 3. burn rate ----------
def burn_rate(errors, elapsed_fraction):
    consumed = errors / error_budget # fraction of budget spent
    return consumed / elapsed_fraction, consumed

print("== burn rate, 10 days into the month ==")
elapsed = 10 / 30 # 1/3 of the window
for errors in (300, 600):
    br, consumed = burn_rate(errors, elapsed)
    projected = errors / elapsed # month-end errors at this pace
    flag = "OK" if br <= 1 else "ALERT"
    print(f" errors={errors:4d} consumed={consumed:.2f} "
          f"burn={br:.2f} projected={projected:.0f} [{flag}]")

# ---------- 4. multi-window fast burn ----------
print("== fast-burn (1-hour window) ==")
hour_frac = 1 / (30 * 24) # 1 hour as a fraction of month
errors_hr = 50
br_hr, _ = burn_rate(errors_hr, hour_frac)
hours_to_exhaust = error_budget / errors_hr
print(f" {errors_hr} errors in 1h burn={br_hr:.1f}x "
      f"exhaust in {hours_to_exhaust:.0f} h ({hours_to_exhaust/24:.2f} d)")

# ---------- 5. SEV-1 revenue at risk ----------
print("== revenue at risk if the full month's budget burns as downtime ==")
for A in (0.995, 0.999, 0.9995):
    d = downtime_min(A)
    print(f" A={A*100:6.2f}% {d:7.1f} min -> ${d*REV_PER_MIN:,.0f}")

# Expected output:
# == allowed downtime per 30-day month ==
# A= 99.50% downtime = 216.0 min
# A= 99.90% downtime = 43.2 min
# A= 99.95% downtime = 21.6 min
# == error budget == 1000 failed trajectories / month
# == burn rate, 10 days into the month ==
# errors= 300 consumed=0.30 burn=0.90 projected=900 [OK]
# errors= 600 consumed=0.60 burn=1.80 projected=1800 [ALERT]
# == fast-burn (1-hour window) ==
# 50 errors in 1h burn=36.0x exhaust in 20 h (0.83 d)
# == revenue at risk if the full month's budget burns as downtime ==
# A= 99.50% 216.0 min -> $43,200
# A= 99.90% 43.2 min -> $8,640
# A= 99.95% 21.6 min -> $4,320
```

— end of document —